



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

There slides are mainly extracted from:

Frank Tsui, Orlando Karam, Barbara Bernal, Software engineering (Third Edition), Jones & Bartlett Learning

Robert C. Martin: Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall; 2017

Neal Ford, Mark Richards: Fundamentals of Software Architectures, O'Reilly 2021

Raju Gandhi, Mark Richards, Neal Ford: Head First Software Architecture: A Learner's Guide to Architectural Thinking 1st Edition, 2024

Software architecture and design 1

Francesco Guerra
francesco.guerra@unimore.it

OBJECTIVES

- ▶ Design: Architecture and Methodology
- ▶ Understand the differences between architectural and detailed design.
- ▶ Understand the SOLID principles for the OOP, and the other principles for evaluating component and architecture design.
- ▶ Gain knowledge of common software architectural styles, tactics, and reference architectures.
- ▶ Gain knowledge of basic techniques for detailed design, including functional decomposition, relational database design, and object-oriented design.

Software architecture and software design

- ▶ Every software system delivers two values to stakeholders:
 - ▶ **Behavior** → the system does what is required
 - ▶ **Structure** → the system can be understood, maintained, and evolved
- ▶ Developers must preserve **both values**
 - ▶ Risk: focusing only on one while neglecting the other
- ▶ Behavior in practice:
 - ▶ Programmers develop implementing functional specifications.
 - ▶ Code makes machines meet stakeholder requirements
 - ▶ Violations → bugs are fixed

Software design and architecture

- ▶ Structure
 - ▶ Software was invented to be “soft.”
 - ▶ It was intended to be a way to easily change the behavior of machines.
 - ▶ To fulfill its purpose, software must be soft: it must be easy to change.
 - ▶ When stakeholders change their minds → adapting should be easy.
 - ▶ Effort should depend on the **scope** of the change, not its **shape**
 - ▶ Year 1 < Year 2 < Year 3 (in cost and effort)
 - ▶ **Root cause:** problems in the **system architecture**

Software design and architecture

- ▶ Which provides greater value: behavior or structure?
 - ▶ Is it more important for a software system to work, or to be easy to change?
- ▶ For business managers and developers, the immediate priority is that the system works (**behavior**).
 - ▶ However, a program that works perfectly but cannot be changed will soon become useless.
- ▶ Conversely, if a program does not work but is **easy to change**, a programmer can fix it and keep it working as requirements evolve.
 - ▶ The program remains continually useful.
- ▶ Since code can always be changed, the real challenge is cost: when the **cost of change** exceeds the **value of change**, the system loses its usefulness.
 - ▶ Unfortunately, many systems eventually reach that point.

Software design and architecture

- ▶ If software is used → it will be changed
 - ▶ Architecture's goal: **minimize human effort** in building & maintaining the system
- ▶ The measure of architecture/design quality is simply the measure of the effort required to meet the needs of the customer.
 - ▶ Good design → effort stays low over time
 - ▶ Bad design → effort grows with each release

- ▶ Once requirements are clear, they are transformed into **design**
- ▶ Design defines how the software is structured:
 - ▶ What the components are
 - ▶ How they relate to each other
- ▶ **Two phases of design**
 1. **Architectural design**
 - ▶ High-level overview: main components, external properties, relationships
 - ▶ Driven by functional & non-functional requirements
 1. **Detailed design**
 - ▶ Components decomposed to finer detail
 - ▶ Driven by architecture & functional requirements
 - ▶ All functional requirements must be addressed

Design (2)

- ▶ Traditional processes:
 - ▶ Design documented in full detail
 - ▶ Programmers translate design into code
- ▶ Agile & modern processes:
 - ▶ Programmers play a key role in detailed design
 - ▶ They often create the detailed design while coding
- ▶ Design specification:
 - ▶ Many ways to specify a design
 - ▶ Graphical notations are desirable
 - ▶ UML = standard for object-oriented design

SOLID principles



Characterizing Design OOP

The SOLID Principles (Robert C. Martin).

- ▶ The Single Responsibility Principle (SRP)
 - ▶ A class should have one, and only one, reason to change.
- ▶ The Open Closed Principle (OCP)
 - ▶ You should be able to extend a classes behavior, without modifying it.
- ▶ The Liskov Substitution Principle (LSP)
 - ▶ Derived classes must be substitutable for their base classes.
- ▶ The Interface Segregation Principle (ISP)
 - ▶ Make fine grained interfaces that are client specific.
- ▶ The Dependency Inversion Principle (DIP)
 - ▶ Depend on abstractions, not on concretions.

The Single Responsibility Principle

► What is a Responsibility?

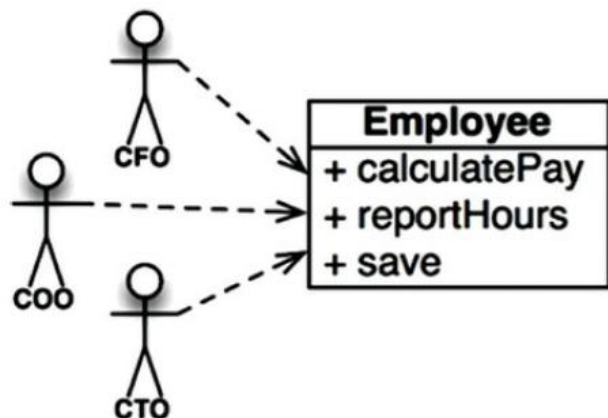
- ▶ A responsibility = “a reason to change”
- ▶ More than one reason to change a class → class has multiple responsibilities
- ▶ Separate responsibilities only if changes actually occur along different axes
- ▶ Single Responsibility Principle (SRP): simple in concept, hard to apply
- ▶ Natural tendency: conjoin responsibilities
- ▶ Software design = finding and separating responsibilities

The Single Responsibility Principle

- ▶ Why is it important to separate responsibilities into separate classes?
 - ▶ Each responsibility = an axis of change
 - ▶ Multiple responsibilities in a class = multiple reasons to change
 - ▶ Coupled responsibilities → changes in one can break others
 - ▶ Leads to fragile, hard-to-maintain designs
 - ▶ Software evolves to satisfy users/stakeholders
 - ▶ A module should serve one and only one user or stakeholder

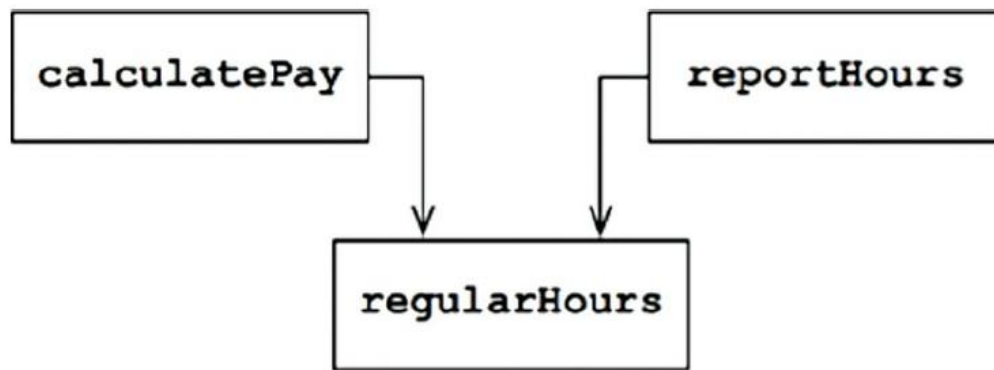
The Single Responsibility Principle

- ▶ What is a “module”?
 - ▶ Simplest definition: a source file
 - ▶ Cohesion = how closely a module supports a single responsibility
 - ▶ Modules/classes with high cohesion:
 - ▶ Follow the Single Responsibility Principle
 - ▶ Are easier to maintain
 - ▶ Modules/classes with low cohesion:
 - ▶ Take on multiple responsibilities
 - ▶ Are harder to maintain



Accidental Duplication

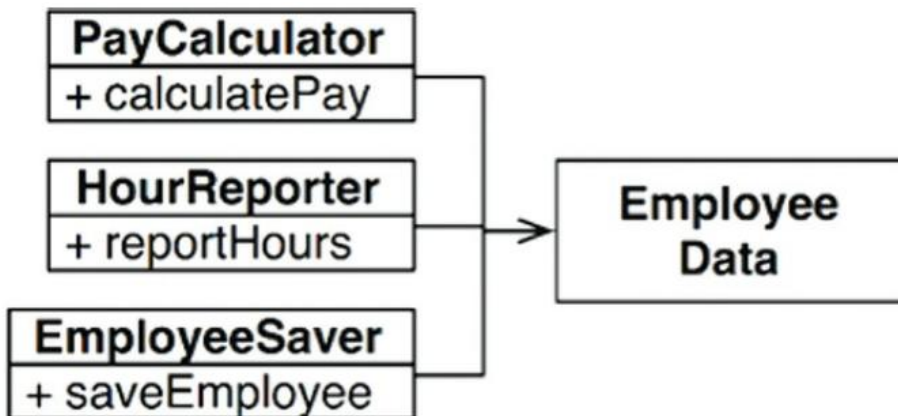
- ▶ Employee class violates SRP:
 - ▶ three methods serve three different actors
 - ▶ calculatePay() → accounting department (CFO)
 - ▶ reportHours() → HR department (COO)
 - ▶ save() → database administrators (CTO)
- ▶ By combining these methods in one class:
 - ▶ Responsibilities become coupled
 - ▶ Each actor is indirectly dependent on the others
- ▶ Result: fragile design, harder to maintain and evolve



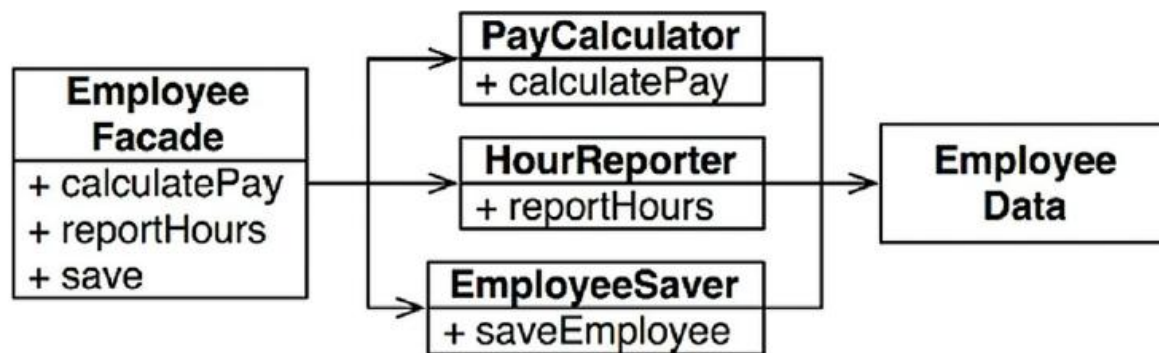
Accidental Duplication

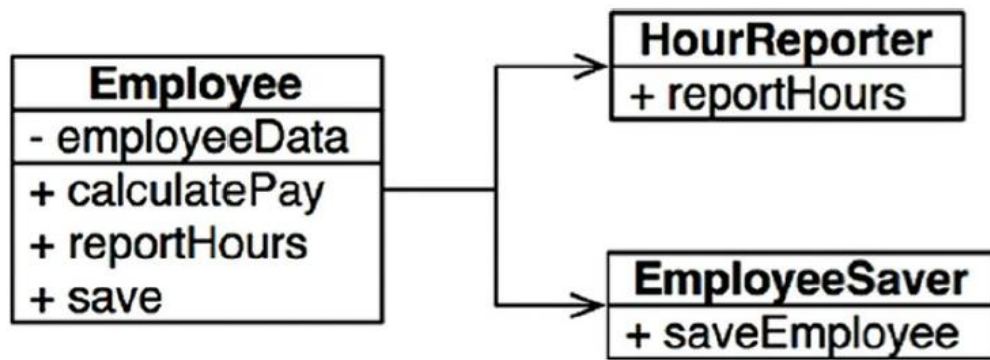
- ▶ Suppose that `calculatePay()` and `reportHours()` share `regularHours()`
- ▶ CFO requests a change to non-overtime hours calculation
 - ▶ Developer updates `regularHours()` to satisfy CFO requirements
 - ▶ Developer tests only `calculatePay()` → change works for CFO
- ▶ Unintended effect: `reportHours()` (used by HR/COO) also calls `regularHours()`
 - ▶ HR continues using reports without knowing the change occurred
- ▶ Result: hidden side effects due to coupled responsibilities

Solution



- ▶ Each class holds only the source code necessary for its particular function.
 - ▶ The three classes are not allowed to know about each other.
 - ▶ Any accidental duplication is avoided.
- ▶ Developers now have three classes that they have to instantiate and track.
 - ▶ A common solution is to use the *Facade* pattern





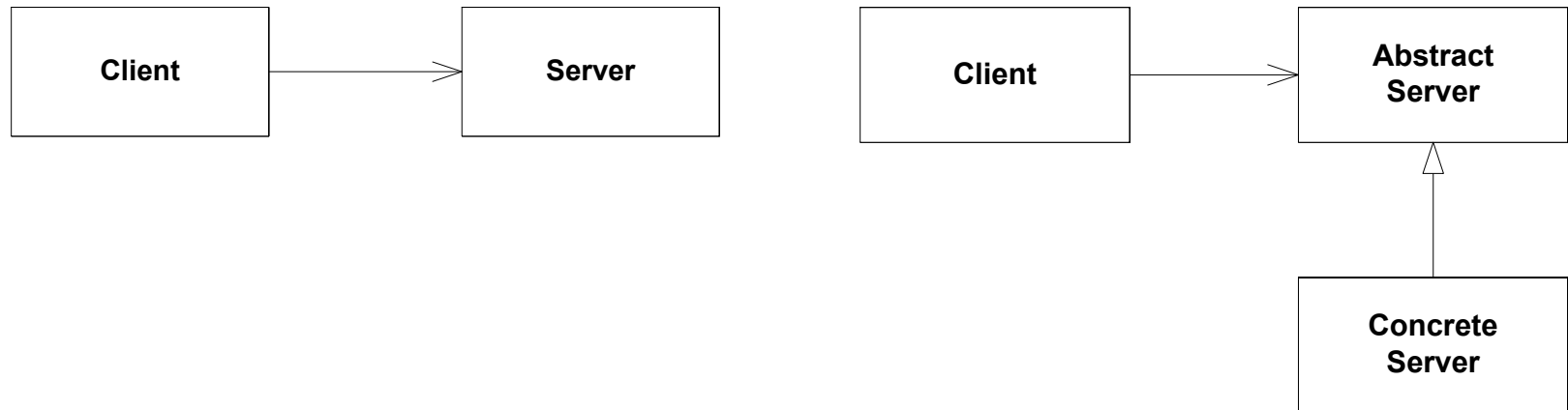
Solution (2)

- ▶ Some developers keep key business rules in the original class (e.g., Employee)
- ▶ Use the class as a Facade for lesser functions
- ▶ Concern: “Does each class end up with only one method?”
 - ▶ Not true: each main responsibility usually requires a family of related methods
 - ▶ Example: salary calculation may involve multiple functions: `calculateBasePay()`, `calculateBonus()`, `applyTaxes()`
- ▶ Each class encapsulates a **family of related methods** → outside the scope, private members are hidden
- ▶ Benefits:
 - ▶ Important business rules stay close to the data
 - ▶ Easier to maintain, extend, and test

The Open Closed Principle

- ▶ Problem:
 - ▶ A single change in a program can trigger a cascade of changes → fragile, rigid, unpredictable, unreusable modules
- ▶ OCP Solution:
 - ▶ Design modules that never need to be modified
 - ▶ When requirements change, extend behavior by adding new code instead of changing existing code
- ▶ Module characteristics:
 1. Open for Extension: module behavior can be extended to meet new requirements
 2. Closed for Modification: existing source code should not be altered
- ▶ Benefit:
 - ▶ Reduces ripple effects of change
 - ▶ Improves maintainability, stability, and reusability

The Open Closed Principle



- ▶ The normal way to extend the behavior of a module is to change it. A module that cannot be changed is normally thought to have a fixed behavior. How can these two opposing attributes be resolved?
- ▶ Abstraction is Key
 - ▶ Client/Server relationships are “open”
 - ▶ Changes to servers cause changes to clients
 - ▶ Abstract servers “close” clients to changes in implementation.

The Open Closed Principle

- ▶ Strategic closure
 - ▶ No significant program can be **100% closed**
 - ▶ **Closure must be strategic**
 - ▶ Designer chooses which changes to guard against
- ▶ Experienced designers consider:
 - ▶ User needs
 - ▶ Industry trends
 - ▶ Likelihood of different kinds of changes
- ▶ Apply the Open-Closed Principle for the most probable changes

Example

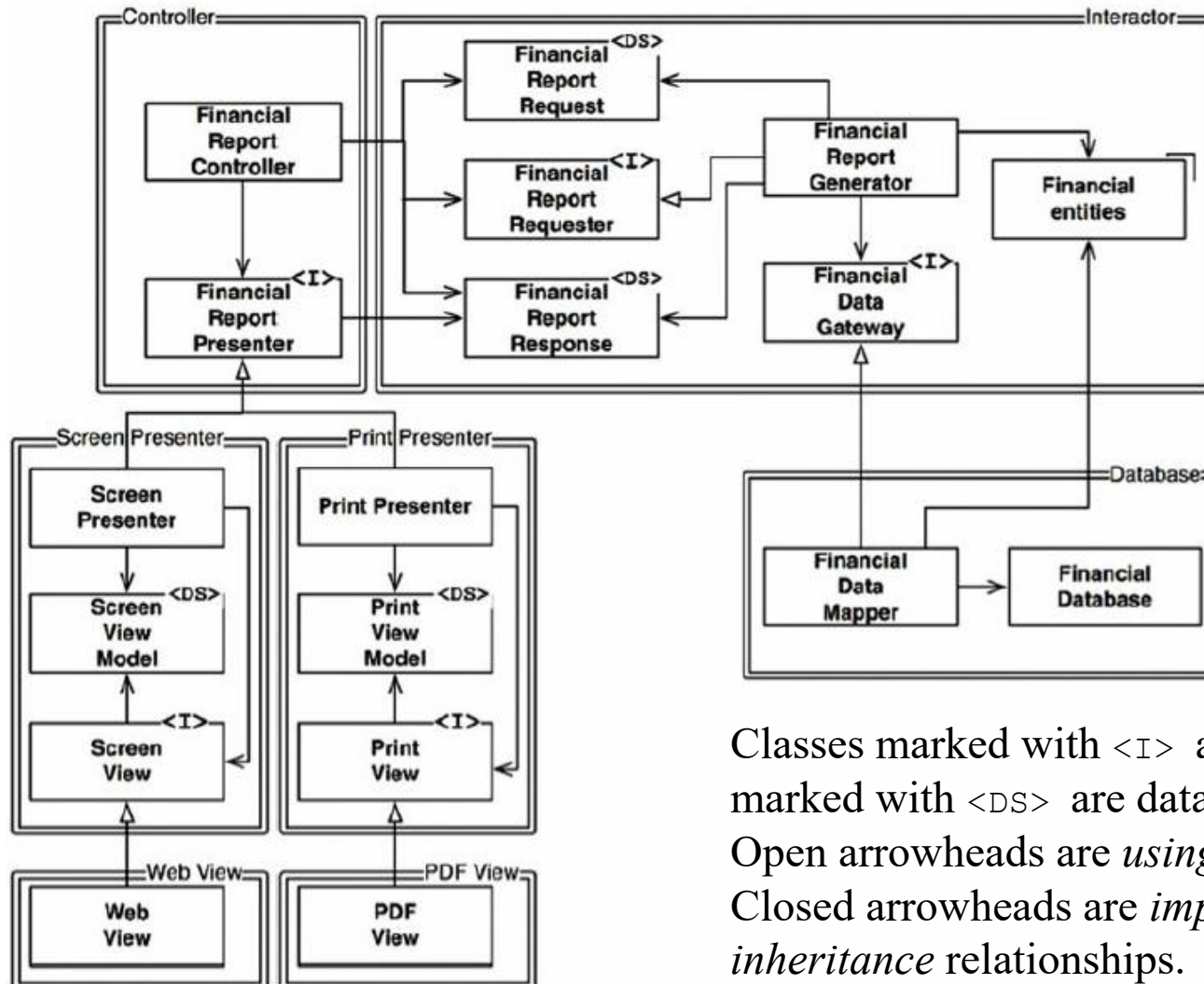
- ▶ We have a system that displays a financial summary on a web page.
 - ▶ Current behavior: **negative numbers in red**
- ▶ The stakeholders ask that this information can be printed on a black-and-white printer.
 - ▶ Negative numbers should be surrounded by parentheses.
- ▶ **Good architecture minimizes changes**
 - ▶ Separate elements that change for different reasons (**SRP**)
 - ▶ Organize dependencies properly between these elements
- ▶ **Benefit:**
 - ▶ Only the code that must change is modified
 - ▶ Reduces risk of unintended side effects



Example

- ▶ Generating the report involves **two separate responsibilities**:
 1. Calculation of reported data
 2. Presentation of that data
- ▶ **Key principle**: organize dependencies so that changes in one responsibility do not affect the other
- ▶ Goal: enable **behavior extension** without modifying existing code

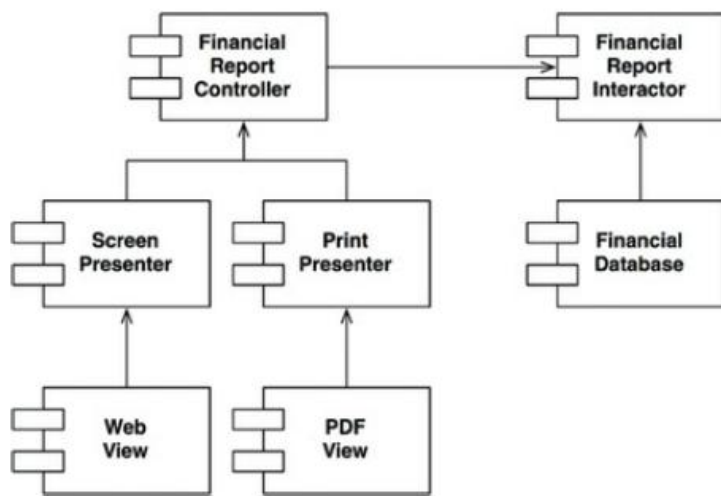
Example



Classes marked with `<I>` are interfaces; those marked with `<DS>` are data structures. Open arrowheads are *using* relationships. Closed arrowheads are *implements* or *inheritance* relationships.

Example

- ▶ All the arrows are *source code* dependencies.
 - ▶ An arrow pointing from class A to class B means that the source code of class A mentions the name of class B, but class B mentions nothing about class A.
 - ▶ `FinancialDataMapper` **knows about** `FinancialDataGateway` through an *implements* relationship, but `FinancialGateway` knows nothing at all about `FinancialDataMapper`.
- ▶ The next thing to notice is that each double line is crossed *in one direction only*.
 - ▶ This means that all component relationships are unidirectional.
 - ▶ These arrows point toward the components that we want to protect from change.



Example

If component A should be protected from changes in component B, then component B should depend on component A.

- ▶ Goal: shield higher-level components from changes in lower-level components
 - ▶ Protect Controller from Presenters
 - ▶ Protect Presenters from Views
 - ▶ Protect Interactor from everything else
- ▶ Interactor:
 - ▶ Contains business rules and highest-level policies
 - ▶ Changes in Database, Controller, Presenters, or Views do not affect it
 - ▶ Deals with the central concern; other components handle peripheral concerns
- ▶ Hierarchy of protection:
 - ▶ Components separated based on how, why, and when they change
 - ▶ Higher-level components are insulated from changes in lower-level components
- ▶ Follows the Open-Closed Principle

Example

- ▶ **Information Hiding**
- ▶ **FinancialReportRequester interface:**
 - ▶ Protects the **Controller** from knowing the internals of the **Interactor**
- ▶ Without this interface:
 - ▶ Controller would have **transitive dependencies** on FinancialEntities
 - ▶ Violates principle: software entities should not depend on things they **don't directly use**
- ▶ **Goal:**
 - ▶ First priority → protect **Interactor** from changes in Controller
 - ▶ Also → protect **Controller** from changes in Interactor by **hiding internals**

The Open Closed Principle

- ▶ OCP is the root motivation behind many OOP heuristics and conventions:
 - ▶ Make all member variables private
 - ▶ No global variables
 - ▶ Runtime type identification is dangerous
- ▶ Benefits of OCP:
 - ▶ Maximizes reusability and maintainability
 - ▶ Not automatic in OOP languages → requires conscious application
- ▶ Key mechanisms:
 - ▶ Abstraction → hide implementation details
 - ▶ Polymorphism → extend behavior without modifying existing code

The Liskov Substitution Principle

- ▶ The principle defines that objects of a superclass shall be replaceable with objects of its subclasses without breaking the application. That requires the objects of your subclasses to behave in the same way as the objects of your superclass.

“What is wanted here is something like the following substitution property: If for each object $o1$ of type S there is an object $o2$ of type T such that for all programs P defined in terms of T , the behavior of P is unchanged when $o1$ is substituted for $o2$ then S is a subtype of T .”

- ▶ Possible rules to avoid it
 - ▶ An overridden method of a subclass needs to accept the same input parameter values as the method of the superclass.
 - ▶ The return value of a method of the subclass needs to comply with the same rules as the return value of the method of the superclass.

The Liskov Substitution Principle

Bad example

```
public class Bird{
    public void fly(){}
}
public class Duck extends Bird{}
```

The duck can fly because of it is a bird, But what about this:

```
public class Ostrich extends Bird{}
```

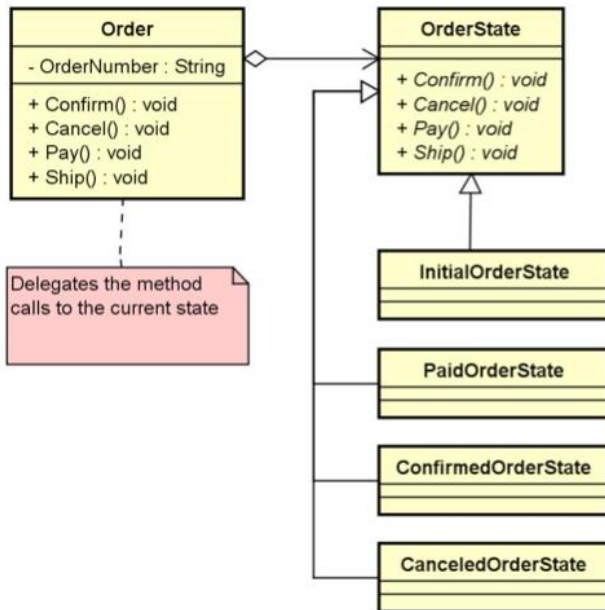
Good example

```
public class Bird{
}
public class FlyingBirds extends Bird{
    public void fly(){}
}
public class Duck extends FlyingBirds{}
public class Ostrich extends Bird{}
```

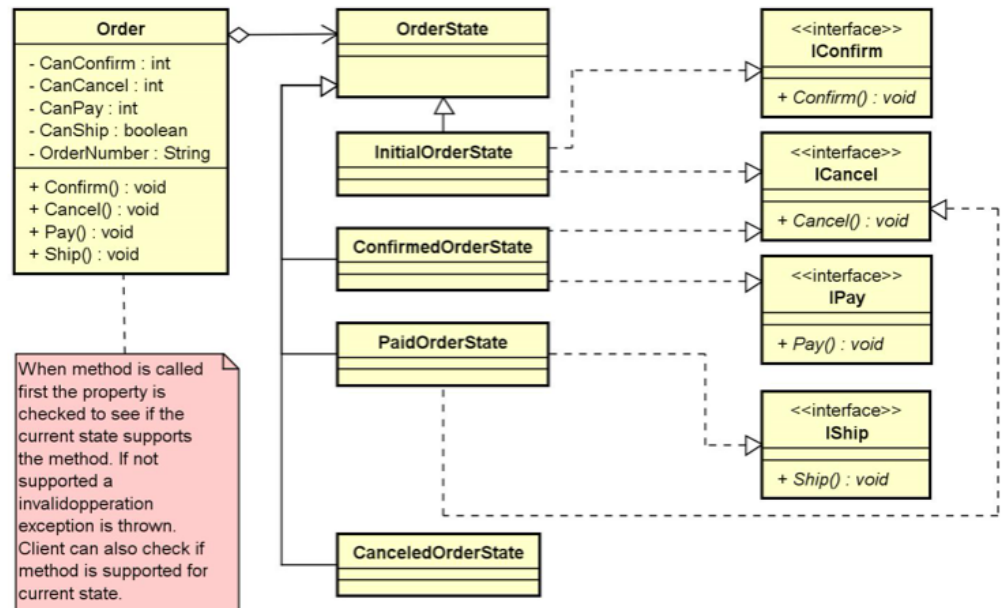


The Liskov Substitution Principle

Violation



Compliance

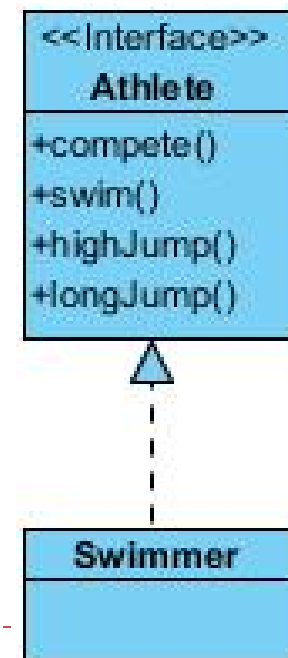


The Liskov Substitution Principle

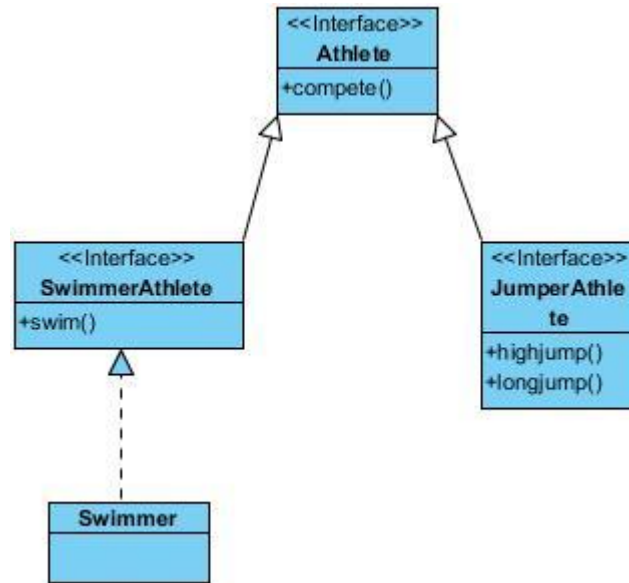
- ▶ Key insight:
 - ▶ A model cannot be validated in isolation
 - ▶ Its validity depends on its clients
- ▶ Design evaluation:
 - ▶ Consider whether the design meets reasonable assumptions made by its users
- ▶ LSP in OOP:
 - ▶ The ISA relationship refers to behavior, not private implementation
 - ▶ Focus on extrinsic public behavior that clients depend on
 - ▶ Ensures subclasses can replace their base classes without breaking client expectations

The Interface Segregation Principle

- ▶ Clients should not be forced to depend upon interfaces that they do not use.
- ▶ Similar to the Single Responsibility Principle, the goal of the Interface Segregation Principle is to reduce the side effects and frequency of required changes by splitting the software into multiple, independent parts.
- ▶ Example
 - ▶ The swimmer has to implement all method even if some of them have no sense for a swimmer

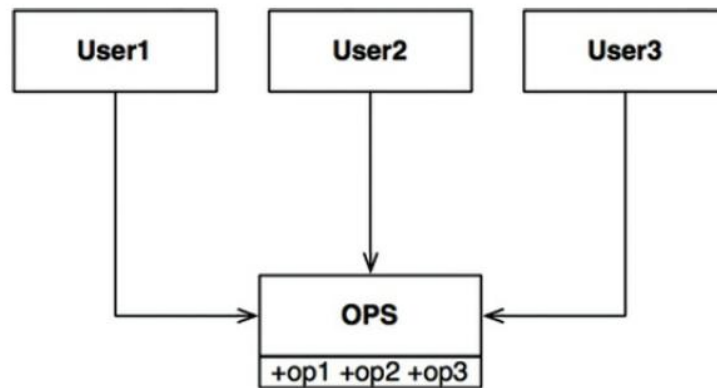


The Interface Segregation Principle



- ▶ Instead of having a unique big interface we should implement smaller and more specific interfaces

Example



Problem:

- Class OPS has multiple operations: op1, op2, op3

Users:

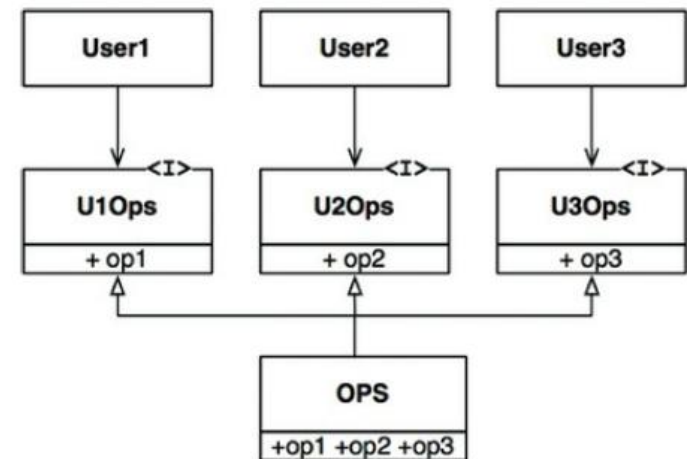
- User1 → only uses op1
- User2 → only uses op2
- User3 → only uses op3

- Issue: User1's code depends on op2 and op3 indirectly

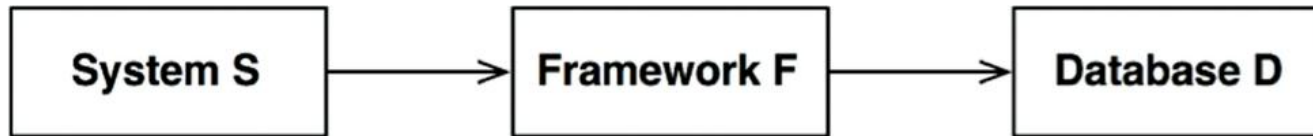
- Changes to op2 force recompilation/ redeployment of User1

Solution:

- Segregate operations into separate interfaces
- Users depend only on the operations they actually use



Example



- ▶ **Principle:** Do not depend on modules that contain more than you need
- ▶ **Source code level:** unnecessary dependencies → forced **recompilation/redeployment**
- ▶ **Architectural level:** same problem at higher scale
 - ▶ Example: System S uses framework F
 - ▶ F is bound to database D
 - ▶ S depends on F → indirectly depends on D
 - ▶ D contains features unused by F and S
 - ▶ **Consequences:**
 - ▶ Changes in unused features → redeployment of F and S
 - ▶ Failures in unused features → may break F and S
- ▶ **Takeaway:** Minimize dependencies to **only what is necessary**

The Dependency Inversion Principle

- ▶ Problem:

- ▶ High-level modules (complex logic) should be reusable and stable
- ▶ Low-level modules (utilities) often change
- ▶ If high-level depends directly on low-level → fragile design

- ▶ Principle:

- ▶ High-level modules should not depend on low-level modules
- ▶ Both should depend on abstractions
- ▶ Abstractions must not depend on details
Details (low-level) depend on abstractions

- ▶ Solution:

- ▶ Introduce an abstraction layer between high-level and low-level
- ▶ High-level depends on the abstraction
- ▶ Low-level also depends on the same abstraction

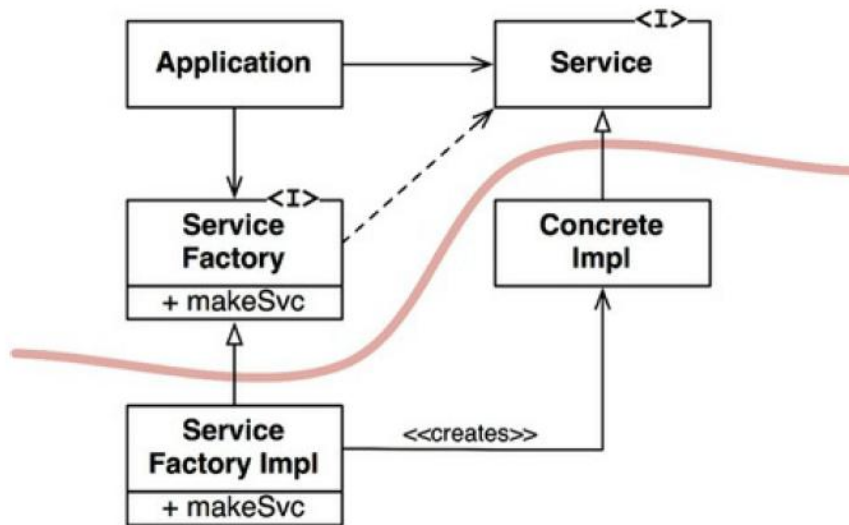
Stable Abstractions vs. Volatile Concretions

- ▶ Interfaces are more stable than implementations
 - ▶ Changing an interface → impacts all implementations
 - ▶ Changing an implementation → does not affect the interface
- ▶ Implication for architecture:
 - ▶ Depend on stable abstract interfaces
 - ▶ Avoid dependence on volatile concretions
- ▶ Coding practices:
 - ▶ Don't refer to volatile concrete classes → use Abstract Factories
 - ▶ Don't derive from volatile concrete classes → inheritance is rigid
 - ▶ Don't override concrete functions → instead, make them abstract and provide multiple implementations

```
class DatabaseConnection {  
    public void connect() {  
        System.out.println("Connecting to SQL Database");  
    }  
}  
  
class Service {  
    private DatabaseConnection dbConnection;  
  
    public Service() {  
        // Directly referencing a concrete class  
        this.dbConnection = new DatabaseConnection();  
    }  
  
    public void performService() {  
        dbConnection.connect();  
    }  
}
```

Don't refer to volatile concrete classes

- ▶ Service class is tightly coupled to DatabaseConnection.
 - ▶ If you decide to change DatabaseConnection to support a different database (e.g., NoSQL or cloud-based), you would have to modify all places where it is directly referenced.
 - ▶ This leads to code fragility and makes it hard to change the DatabaseConnection class without breaking other parts of the system.



Factories

- ▶ To comply with these rules, the creation of volatile concrete objects requires special handling.

- ▶ This caution is warranted because, in virtually all languages, the creation of an object requires a source code dependency on the concrete definition of that object.

▶ The Solution: Abstract Factory

- ▶ Introduce a **Service interface** → stable contract
- ▶ Introduce a **ServiceFactory interface** → stable creation mechanism
- ▶ Application depends **only on abstractions** (Service, ServiceFactory)

▶ How It Works

- ▶ **Application** needs a **Service**
- ▶ Calls **ServiceFactory.makeSvc()**
- ▶ **ServiceFactoryImpl** (concrete factory) implements the creation logic
- ▶ Instantiates **ConcreteImpl** (volatile)
- ▶ Returns it as **Service** (stable abstraction)
- ▶ Application never sees the **ConcreteImpl**

Example Abstract Factory

```
// Abstraction
interface SenderMessage {
    void send();}

// Concrete implementation
class TelegramSenderMessage implements SenderMessage {
    public void send() {
        System.out.println("Sending a telegram");    }}

// Factory abstraction
interface SenderMessageFactory {
    SenderMessage createSender();
}

// Concrete factory
class TelegramSenderMessageFactory implements SenderMessageFactory {
    public SenderMessage createSender() {
        return new TelegramSenderMessage();
    }
}
```


Example Abstract Factory

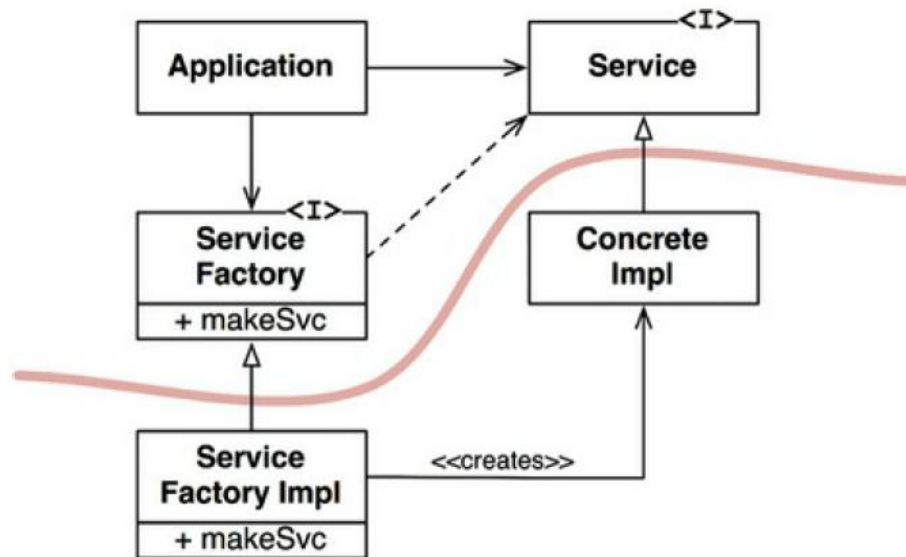
```
// High-level service depending on abstraction
class Service {
    private SenderMessage senderMessage;

    public Service(SenderMessageFactory factory) {
        this.senderMessage = factory.createSender();
    }

    public void performService() {
        senderMessage.send();
    }
}

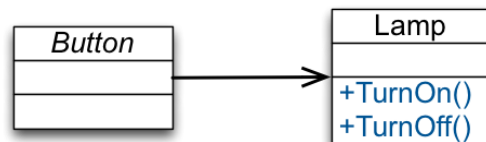
// Test
public class Main {
    public static void main(String[] args) {
        SenderMessageFactory factory = new
TelegramSenderMessageFactory();
        Service service = new Service(factory);
        service.performService();
    }
}
```

Factories



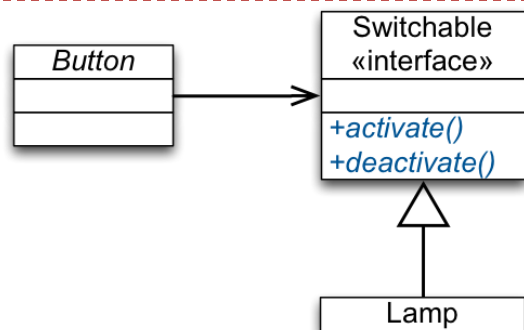
- ▶ The curved line = architectural boundary separating abstract from concrete
- ▶ Source code dependencies always point toward the abstract side
- ▶ Abstract component: contains high-level business rules
- ▶ Concrete component: contains implementation details manipulated by business rules
- ▶ Flow of control crosses the line opposite to dependencies
- ▶ Dependencies are inverted relative to control flow → this is why it's called Dependency Inversion

The Dependency Inversion Principle



- ▶ We cannot reuse *Button* since it depends directly on *Lamp*. (But there are plenty of other uses for *Button*.)
- ▶ *Button* should not depend on the details represented by *Lamp*.
- ▶ The high-level policy (detection of on/off gestures) underlying this (mini) design depends on the low-level details.
- ▶ The underlying abstraction is the detection of on/off gestures and their delegation to a server object that can handle them.
- ▶ If the interface of *Lamp* is changed, *Button* has to be at least tested or even adjusted, even though the policy that *Button* represents is not changed!
- ▶ To make the high-level policy independent of details we should be able to define it independent of the details of *Lamp* or any other specific device.

The Dependency Inversion Principle



- ▶ Now *Button* only depends on abstractions; it can be reused with various classes that implement *Switchable*.
- ▶ Changes in *Lamp* will not affect *Button*.
- ▶ The dependencies have been inverted: *Lamp* now has to conform to the interface defined by *Button*.
- ▶ Actually: both depend on an abstraction!

And not only ...

- ▶ Dependency injection
 - ▶ The general concept behind dependency injection is called Inversion of Control.
 - ▶ A class should not configure its dependencies statically but should be configured from the outside.
 - ▶ Ideally Java classes should be as independent as possible from other Java classes.
 - ▶ This increases the possibility of reusing these classes and to be able to test them independently from other classes.
 - ▶ If the Java class creates an instance of another class via the new operator, it cannot be used (and tested) independently from this class and this is called a hard dependency.

Dependency injection

```
public class MyClass {  
    private Logger logger;  
    public MyClass(Logger logger) {  
        this.logger = logger;  
        // write an info log message  
        logger.info("This is a log message.")  
    }  
}
```

- ▶ This way the Java class has no hard dependencies, which means it does not rely on an instance of a certain class. This allows you to test your class in isolation.
- ▶ If dependency injection is used, a Java class can be tested in isolation.
- ▶ There are at least three ways an object can receive a reference to an external module:
 - ▶ constructor injection: the dependencies are provided through a class constructor.
 - ▶ setter injection: the client exposes a setter method that the injector uses to inject the dependency.
 - ▶ interface injection: the dependency provides an injector method that will inject the dependency into any client passed to it. Clients must implement an interface that exposes a setter method that accepts the dependency.

Designing Components



Designing Components

▶ Components

- ▶ Units of deployment: smallest entities that can be deployed as part of a system
- ▶ Examples:
 - ▶ In Java: .jar files
 - ▶ In compiled languages: binary file aggregations
 - ▶ In general they are the granule of deployment
- ▶ Deployment options:
 - ▶ Linked into a single executable
 - ▶ Aggregated into archives (.war, etc.)
 - ▶ Deployed as dynamic plugins (.jar, .dll, .exe)
- ▶ Key property:
 - ▶ Well-designed components remain independently deployable
 - and thus independently developable

Component Cohesion

- ▶ Which classes belong in which components?
- ▶ Three principles of component cohesion can support your choice:
 - ▶ REP: The Reuse/Release Equivalence Principle
 - ▶ CCP: The Common Closure Principle
 - ▶ CRP: The Common Reuse Principle

Reuse/Release Equivalence Principle

- ▶ *The granule of reuse is the granule of release.*
- ▶ Reuse requires a **release process**:
 - ▶ Components must have **release numbers**
 - ▶ Ensures compatibility between components
 - ▶ Developers must know **when releases occur** and **what changes they bring**
- ▶ Design implication:
 - ▶ Classes and modules in a component must form a cohesive group
 - ▶ They should be releasable together
 - ▶ Grouping should make sense to authors and users
- ▶ Key idea:
 - ▶ REP violations are obvious: if grouping “doesn’t make sense,” users will notice
 - ▶ Respecting REP → increases credibility and architectural quality

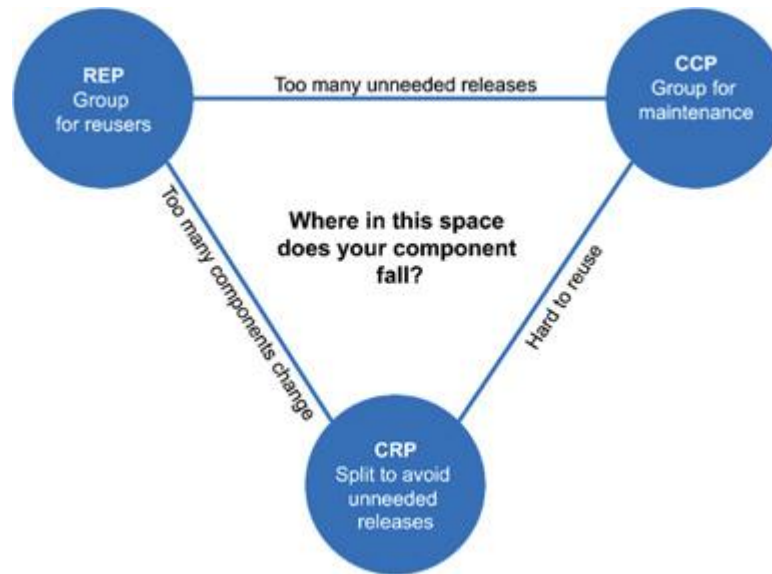
Common Closure Principle

- ▶ *Gather into components those classes that change for the same reasons and at the same times.*
- ▶ Component design rule:
 - ▶ Group classes that change **for the same reasons & at the same times**
 - ▶ Separate classes that change for **different reasons**
- ▶ Analogy with SRP:
 - ▶ SRP: a class = one reason to change
 - ▶ CCP: a component = one reason to change
- ▶ Practical impact:
 - ▶ Maintainability. Reusability (in most systems)
 - ▶ Ideally, all related changes affect only one component
 - ▶ Easier redeployment → only redeploy the modified component
- ▶ Key idea:
 - ▶ Classes that always change together (physically or conceptually) **belong in the same component**

Common Reuse Principle

- ▶ *Don't force users of a component to depend on things they don't need.*
- ▶ Rule:
 - ▶ Classes and modules that are reused together belong in the same component
 - ▶ Classes are rarely reused in isolation
- ▶ Implications:
 - ▶ Classes in the same component naturally depend on each other
 - ▶ If a component depends on another → it depends on all its classes (even if using only one)
 - ▶ Changes in the used component may force recompilation, revalidation, redeployment of the using component
- ▶ Design guidance:
 - ▶ Group only classes that are always reused together
 - ▶ Avoid forcing dependencies on unused classes
 - ▶ CRP = component-level version of ISP (Interface Segregation Principle)

Tension diagram for component cohesion



- ▶ The REP and CCP are *inclusive* principles: Both tend to make components larger.
- ▶ The CRP is an *exclusive* principle, driving components to be smaller. It is the tension between these principles that good architects seek to resolve.
- ▶ Balancing these forces with the needs of the application is nontrivial.
 - ▶ The composition of the components evolves with time as the focus of the project changes from develop-ability to reusability.

Summary

Principle	Focus	Rule	Benefit
REP (Reuse/Release Equivalence Principle)	Reuse	The granule of reuse is the granule of release. Components must be tracked and versioned together.	Ensures consistent and safe reuse through release numbers.
CCP (Common Closure Principle)	Maintainability	Classes that change for the same reasons belong in the same component.	Localizes changes → only one component needs redeployment.
CRP (Common Reuse Principle)	Dependency Management	Classes that are reused together should be grouped; avoid depending on unused classes.	Minimizes unnecessary recompilation and redeployment.



Component Coupling

- ▶ How can we deal with the relationships between components?
- ▶ Three principles can be introduced:
 - ▶ ADP: Acyclic Dependencies Principle
 - ▶ SDP: Stable Dependencies Principle
 - ▶ SAP: Stable Abstraction Principle

Acyclic Dependencies Principle

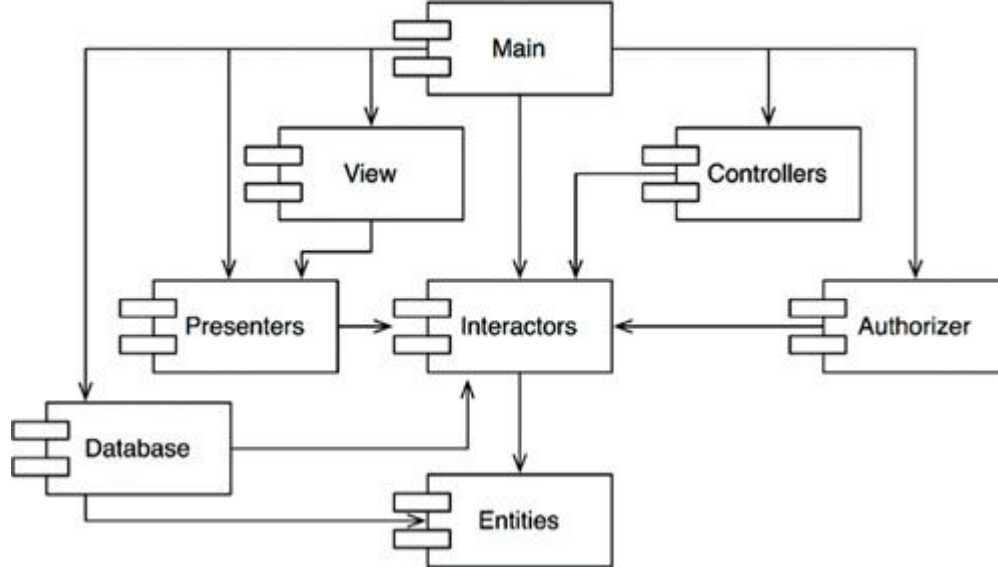
- ▶ *Allow no cycles in the component dependency graph.*
- ▶ When multiple developers modify the same source files, changes can break others' work
 - ▶ Small projects are manageable
 - ▶ Large projects may go weeks without a stable build
 - ▶ Continuous adjustments are needed to keep code compatible with others' changes
- ▶ Two solutions:
 - ▶ Weekly build
 - ▶ Acyclic Dependencies Principle (ADP)

The weekly build

- ▶ Common in medium-sized projects
- ▶ Developers work **in isolation** on private copies for the first four days of the week
- ▶ On Friday, all changes are **integrated and the system is built**
- ▶ **Advantage:** allows developers to work independently
- ▶ **Disadvantage:** large integration penalty on FridayAs projects grow, single-day integration becomes infeasible
 - ▶ Options: 2 days per week (reduces productivity)
 - ▶ Biweekly integration days (also problematic for large projects)
- ▶ Can lead to a project integration crisis

Eliminating Dependency Cycle

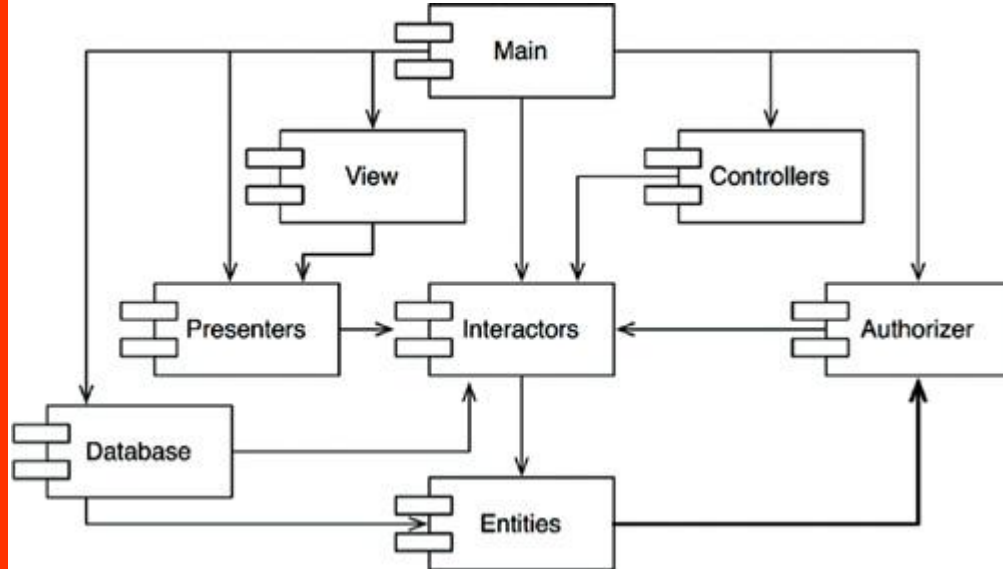
- ▶ Solution: split the development environment into releasable components
- ▶ Components become units of work for individual developer teams
- ▶ Workflow:
 - ▶ Team develops a component → releases it for others to use
 - ▶ Assign a release number and place it in a shared directory
 - ▶ Continue modifying their private copy for the next release
 - ▶ Other teams can choose when to adopt a new release
 - ▶ Continue using old release until ready
- ▶ Benefit: no team is dependent on others' pace
- ▶ Requirement: component dependency structure must be acyclic



Eliminating Dependency Cycle

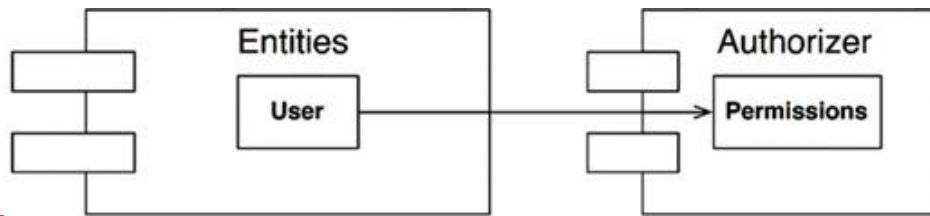
This structure has no cycles. It is a directed acyclic graph (DAG).

- ▶ Example: Presenters component release
 - ▶ Affects View and Main
 - ▶ Main release does not affect other components
- ▶ Testing workflow: Developers build Presenters with current versions of Interactors and Entities
- ▶ System release: bottom-up approach
 1. Compile, test, and release Entities
 2. Compile, test, and release Database and Interactors
 3. Continue upwards until top-level components
- ▶ Benefit: clear, manageable process due to understood dependencies

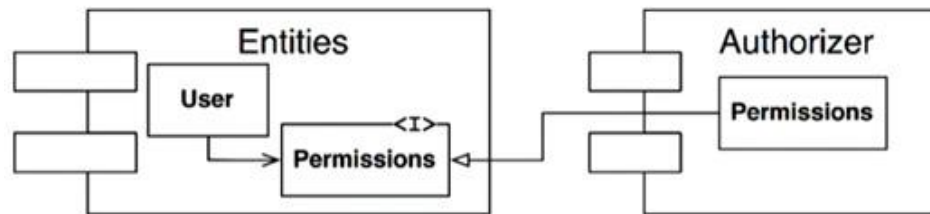


The effect of a cycle

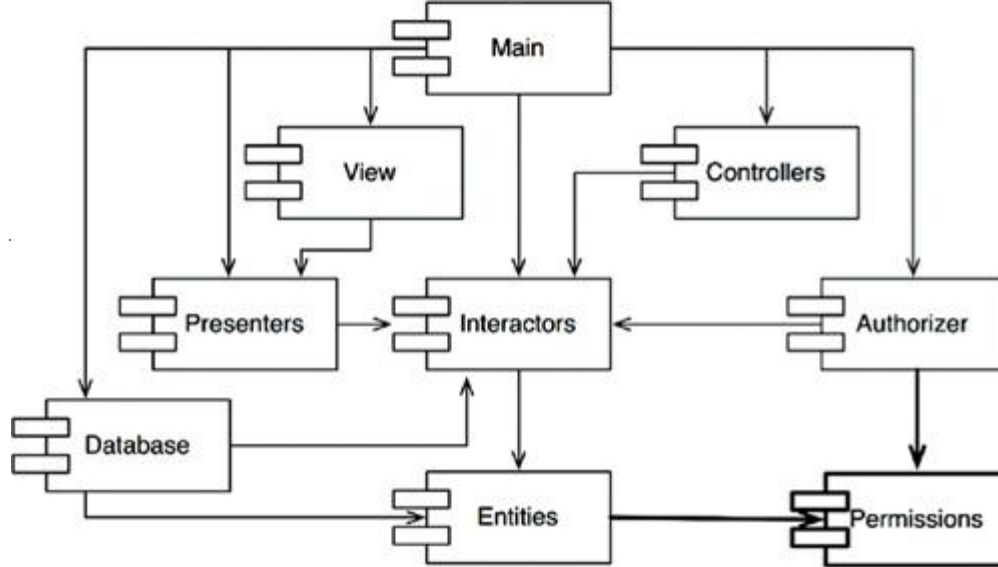
- ▶ This cycle creates some immediate problems.
 - ▶ The developers working on the Database component know that the component must be compatible with Entities which has to be compatible with Authorizer, which depends on Interactors.
 - ▶ This makes Database much more difficult to release.
 - ▶ Entities, Authorizer, and Interactors have, in effect, become one large component.
- ▶ When we want to test the Entities component, we must build and integrate with Authorizer and Interactors.
 - ▶ This level of coupling between components is troubling.



Breaking the cycle



- ▶ Solution 1. Apply the Dependency Inversion Principle (DIP).
- ▶ We could create an interface that has the methods that User needs.
- ▶ We could then put that interface into Entities and inherit it into Authorizer.
- ▶ This inverts the dependency between Entities and Authorizer, thereby breaking the cycle



Breaking the cycle

- ▶ Solution 2. Create a new component that both Entities and Authorizer depend on.
- ▶ Move the class(es) that they both depend on into that new component

Remarks

- ▶ Component structure **cannot be fully designed top-down**; it evolves as the system grows
- ▶ Component decomposition usually mirrors **high-level functional decomposition**
- ▶ Components ideally **represent system functions**
- ▶ Component dependency diagrams **do not inherently reflect functionality**
- ▶ Components serve as a **map for buildability and maintainability**
 - ▶ Not needed at project start (no software to build/maintain yet)
- ▶ Key concern: isolate volatility
 - ▶ Frequently changing components **should not affect stable components**
- ▶ Dependency graph protects **high-value, stable components** from volatile ones

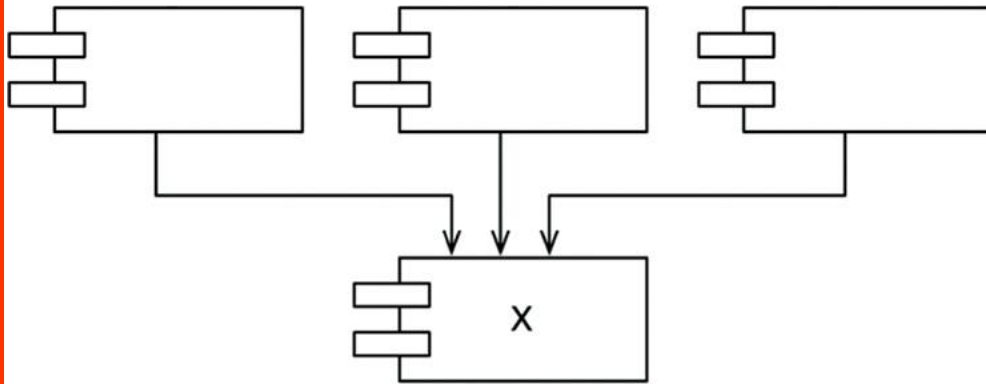
Stable Dependencies Principle

- ▶ *Depend in the direction of stability.*
- ▶ Designs cannot be completely static
 - ▶ Some components are intentionally volatile and expected to change
- ▶ CCP (Common Closure Principle):
 - ▶ Components are sensitive to certain changes but immune to others
- ▶ Key rule:
 - ▶ Volatile components should not be depended on by components that are hard to change
- ▶ Problem:
 - ▶ A module designed to be easy to change can become difficult if others add dependencies on it
- ▶ SDP (Stable Dependencies Principle):
 - ▶ Ensures that easily changeable modules are not depended on by less flexible modules

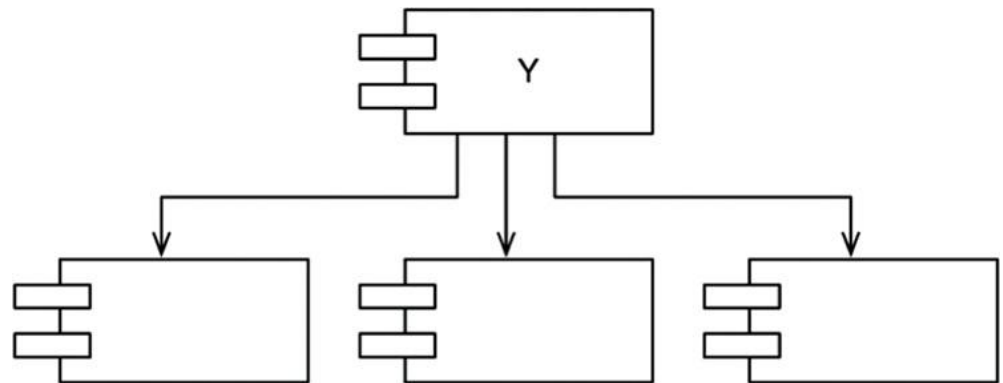
Stability

- ▶ What is meant by “stability”?
 - ▶ Stability is related to the amount of work required to make a change.
- ▶ Many factors may make a software component hard to change
 - ▶ size, complexity, and clarity, among other characteristics.
- ▶ One way to make a software component difficult to change is to make many other software components depend on it.
- ▶ A component with lots of incoming dependencies is very stable
 - ▶ it requires a great deal of work to reconcile any changes with all the dependent components.

Stability



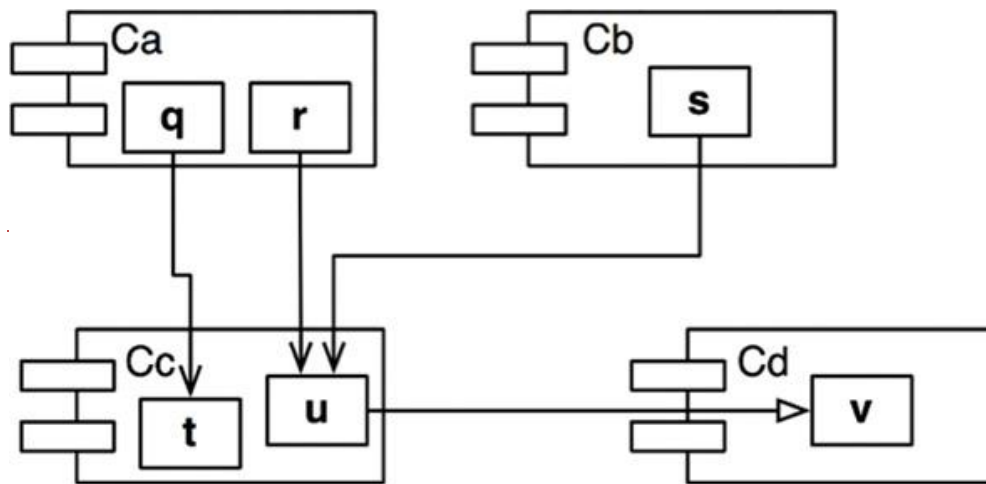
- ▶ X is a stable component
- ▶ Y is a unstable component



Stability Metrics

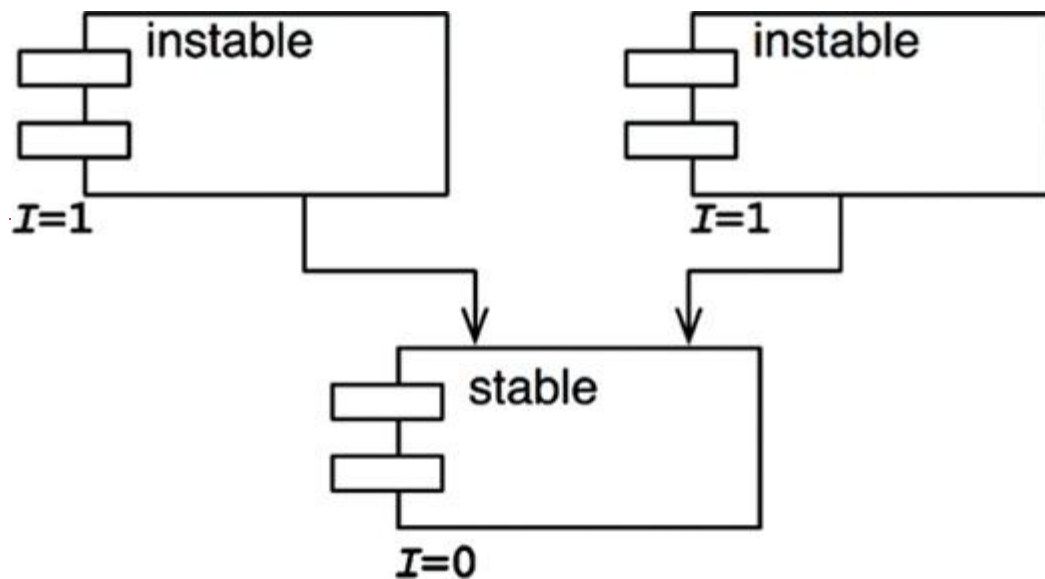
- ▶ One way is to measure the stability of a component is to count the number of dependencies that enter and leave that component.
 - ▶ These counts allow us to calculate the *positional* stability of the component.
- ▶ *Fan-in: Number of classes outside the component that depend on classes inside*
- ▶ *Fan-out: Number of classes inside the component that depend on classes outside*
- ▶ Instability:
 - ▶ Range: $0 \rightarrow 1$
 - ▶ $I = 0$: maximally stable
 - ▶ $I = 1$: maximally unstable

$$I = \frac{\text{Fan-out}}{\text{Fan-in} + \text{Fan-out}}$$



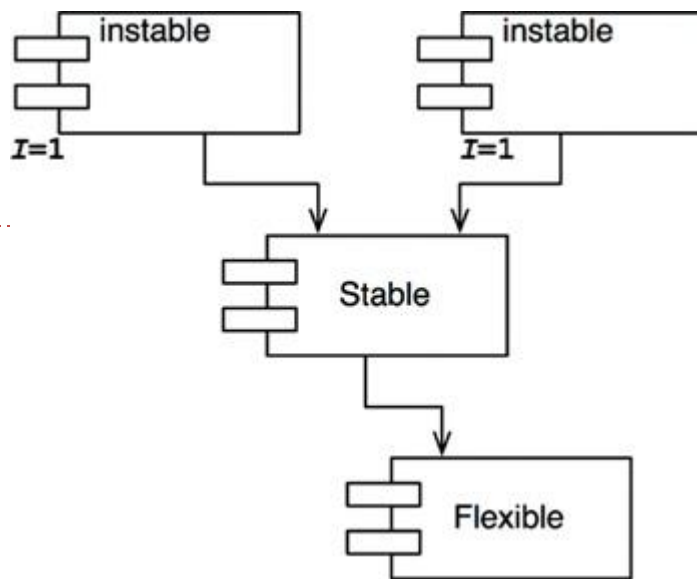
Stability Metrics

- ▶ We want to calculate the stability of the component Cc
 - ▶ Fan-in = 3, Fan-out = 1 and $I = 1/4$.
- ▶ When the $I = \text{Fan-out} / (\text{Fan-in} + \text{Fan-out})$ metric is equal to 1
 - ▶ no other component depends on this component (Fan-in = 0),
 - ▶ and this component depends on other components (Fan-out > 0).
 - ▶ This situation is as unstable as a component can get; it is irresponsible and dependent. Its lack of dependents gives the component no reason not to change, and the components that it depends on may give it ample reason to change.
- ▶ In contrast, when the I metric is equal to 0, it means that
 - ▶ the component is depended on by other components (Fan-in > 0),
 - ▶ but does not itself depend on any other components (Fan-out = 0).
 - ▶ Such a component is responsible and independent. It is as stable as it can get. Its dependents make it hard to change the component, and its has no dependencies that might force it to change.
- ▶ The SDP says that the I metric of a component should be larger than the I metrics of the components that it depends on. That is, I metrics should decrease in the direction of dependency.



Not all components
need to be stable

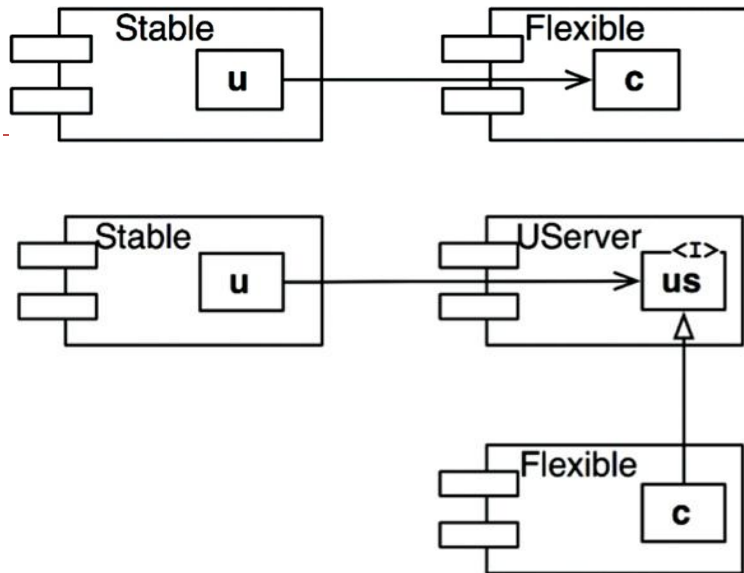
- ▶ If all the components in a system were maximally stable, the system would be unchangeable (undesiderable).
- ▶ Indeed, we want to design our component structure so that some components are unstable and some are stable.
 - ▶ The changeable components are on top and depend on the stable component at the bottom.
 - ▶ Putting the unstable components at the top of the diagram is a useful convention because any arrow that points up is violating the SDP (and, as we shall see later, the ADP)



SDP violated

- ▶ Flexible is a component that we have designed to be easy to change.
 - ▶ We want Flexible to be unstable.
- ▶ However, some developer, working in the component named Stable, has hung a dependency on Flexible.
- ▶ This violates the SDP because the I metric for Stable is much smaller than the I metric for Flexible.
- ▶ As a result, Flexible will no longer be easy to change. A change to Flexible will force us to deal with Stable and all its dependents.

Violation Fixed



- ▶ We can fix this by employing the DIP.
- ▶ We create an interface class called US and put it in a component named UServer.
 - ▶ We make sure that this interface declares all the methods that U needs to use.
 - ▶ We then make C implement this interface.
- ▶ This breaks the dependency of Stable on Flexible, and forces both components to depend on UServer.
- ▶ UServer is very stable ($I = 0$), and Flexible retains its necessary instability ($I = 1$).
- ▶ All the dependencies now flow in the direction of decreasing I .

Stable Abstraction Principle

- ▶ Principle:
- ▶ A component should be as abstract as it is stable
- ▶ Stable components ($I = 0$):
 - ▶ Contain software that rarely changes
 - ▶ Represent high-level architecture and policy
- ▶ Unstable components ($I = 1$):
 - ▶ Contain volatile software, easy to change
- ▶ Challenge:
 - ▶ High-level policies in stable components can be hard to modify → may reduce flexibility
- ▶ Solution (Open Closed Principle):
 - ▶ Use abstract classes to allow extension without modification
 - ▶ Maintains flexibility even in stable components

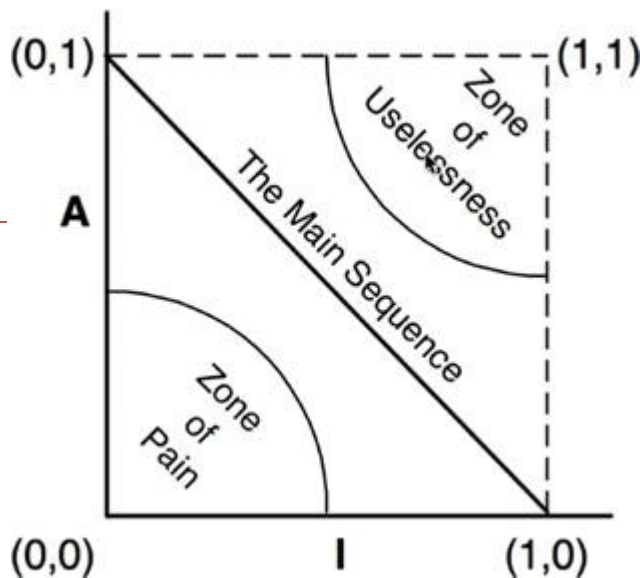
Stable Abstraction Principle

- ▶ Goal: link between stability and abstractness
- ▶ Stable components:
 - ▶ Should be abstract (interfaces, abstract classes)
 - ▶ Allows extension without modification
 - ▶ Maintains flexibility and does not constrain architecture
- ▶ Unstable components:
 - ▶ Should be concrete
 - ▶ Allows easy modification of volatile code
- ▶ Key insight:
 - ▶ Stability implies abstraction
 - ▶ Dependencies flow toward abstraction
- ▶ SAP says that stability implies abstraction. Thus, *dependencies run in the direction of abstraction*.

Measure of the abstraction

- ▶ N_c : The number of classes in the component.
- ▶ N_a : The number of abstract classes and interfaces in the component.
- ▶ A : Abstractness. $A = N_a / N_c$.
- ▶ The A metric ranges from 0 to 1. A value of 0 implies that the component has no abstract classes at all. A value of 1 implies that the component contains nothing but abstract classes.

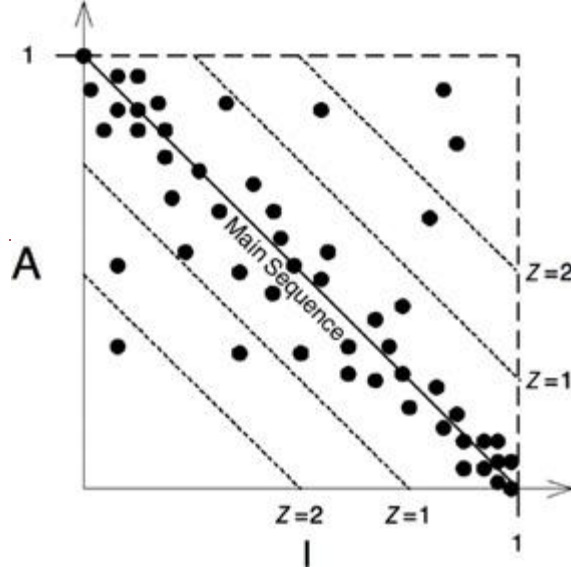
Relations between Stability and Abstraction



- ▶ Zone of Pain (0, 0):
 - ▶ Highly stable and concrete → rigid and difficult to change
 - ▶ Cannot be extended → violates flexibility principles
 - ▶ Often seen in volatile software like database schemas
 - ▶ Nonvolatile concrete components (e.g., String) are harmless, as they rarely change
- ▶ Zone of Uselessness (1, 1):
 - ▶ Maximally abstract but no dependents → no practical use
 - ▶ Often consists of leftover abstract classes never implemented
- ▶ Key insight: Well-designed components avoid (0,0) and (1,1); balance stability, abstraction, and usefulness

Avoiding zones of exclusion

- ▶ Goal: keep volatile components away from zones of exclusion (Pain & Uselessness)
- ▶ Main Sequence: ideal path for component placement
 - ▶ Balances stability and abstractness
 - ▶ Avoids being too abstract for stability or too unstable for abstractness
 - ▶ Components are useful and flexible
- ▶ Endpoints:
 - ▶ Most desirable positions for components
 - ▶ Architects aim to place the majority of components here
- ▶ Other components: Best characteristics if on or near the Main Sequence



Distance from the Main Sequence

- ▶ D : Distance. $D = |A+I-1|$. The range of this metric is $[0, 1]$.
 - ▶ $D = 0$: component is on the Main Sequence; $D = 1$: component is as far as possible from the Main Sequence
- ▶ Given this metric, a design can be analyzed for its overall conformance to the Main Sequence.
 - ▶ The D metric for each component can be calculated. Any component that has a D value that is not near zero can be reexamined and restructured.
- ▶ Statistical analysis of a design is also possible. We can calculate the mean and variance of all the D metrics for the components within a design. We would expect a conforming design to have a mean and variance that are close to zero. The variance can be used to establish “control limits” so as to identify components that are “exceptional” in comparison to all the others.
- ▶ In the scatterplot, we see that the bulk of the components lie along the Main Sequence, but some of them are more than one standard deviation ($Z = 1$) away from the mean. These aberrant components are worth examining more closely. For some reason, they are either very abstract with few dependents or very concrete with many dependents.



References

- ▶ Frank Tsui, Orlando Karam, Barbara Bernal, Software engineering (Third Edition), Jones & Bartlett Learning
- ▶ Robert C. Martin: Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall; 2017
- ▶ Sommerville, Software engineering 9, Pearson
- ▶ Neal Ford, Mark Richards: Fundamentals of Software Architectures, O'Reilly 2021
- ▶ Raju Gandhi, Mark Richards, Neal Ford: Head First Software Architecture: A Learner's Guide to Architectural Thinking 1st Edition, 2024
- ▶ Software Architecture Fundamentals Understanding the Basics
- ▶ Neal Ford, Mark Richards: Fundamentals, Patterns, AntiPatterns, Soft Skills, Continuous Delivery, and Code Analysis Tools, O'Reilly Media March 2014
- ▶ Rachel Roumeliotis: Variations in event-driven architecture
<http://radar.oreilly.com/2015/02/variations-in-event-driven-architecture.html>
- ▶ <https://martinfowler.com/articles/serverless.html>

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.